

Angular

Chapitre 5: Liaison de données et Événements

Enseignant: Mouhamed Amar

Licence 3

Université Amadou Hampaté BA de Dakar

Année 2022-2023



Plan

- Liaison de données: Définition
- Interpolation
- Liaison par propriété
- Liaison d'évènement
- Liaison bidirectionnelle
- Pipe
- Directive
- Partage de données

Liaison de données: **définition**

La liaison de données(**Binding**) maintient automatiquement votre page à jour en fonction de l'état de votre application. Vous utilisez la liaison de données pour spécifier des éléments tels que la source d'une image, l'état d'un bouton ou des données pour un utilisateur particulier.

Interpolation: **définition**

L'interpolation fait référence à l'incorporation d'expressions dans un texte balisé. Par défaut, l'interpolation utilise les doubles accolades `{{` et `}}` comme délimiteurs.

Interpolation: **example**

.html

A screenshot of a web browser window. The address bar shows 'localhost:4200'. The main content area displays the text 'Bonjour Mouhamed' in a large, dark font.

Bonjour Mouhamed

```
<h1>Bonjour {{prenom}}</h1>
```

.ts

```
prenom:string="Mouhamed";
```

A red arrow originates from the variable 'prenom' in the TypeScript code block below and points to the interpolation ' {{prenom}} ' in the HTML code block above.

Interpolation: example

.html

```

```

.ts

```
image_url:string="mouton.jpg";
```

Liaison par propriétés: **définition**

La liaison de propriété(**property binding**) dans Angular vous aide à définir les valeurs des propriétés des éléments HTML ou des directives. Utilisez la liaison de propriété pour effectuer des actions telles que basculer les fonctionnalités des boutons, définir des chemins par programmation et partager des valeurs entre les composants.

Liaison par propriété: **exemple**

La propriété **src** prends comme valeur la variable **image_url**

.html

```
<img alt="image" [src]="image_url">
```

.ts

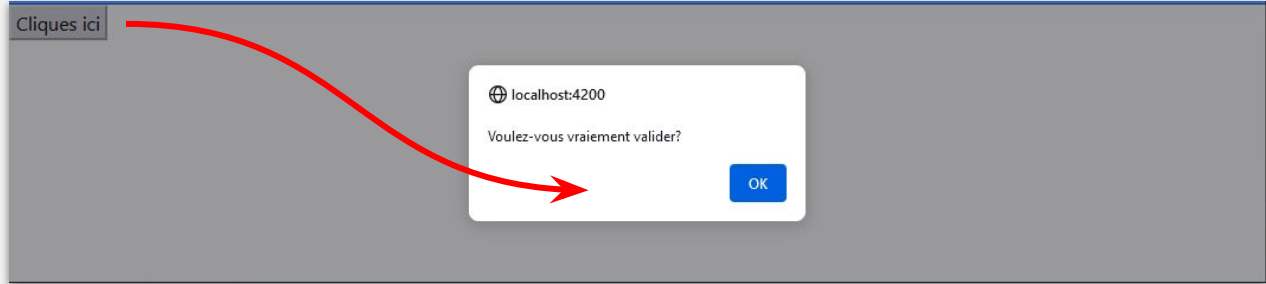
```
image_url:string="mouton.jpg";
```


Liaison d'évènement: **définition**

La liaison d'événements(**Event binding**) vous permet d'**écouter** et de **réagir** aux actions de l'utilisateur telles que les frappes au clavier, les mouvements de la souris, les clics et les touches.

Liaison d'événement: **exemple**

.html



```
<button (click)="valider()">Cliquez ici</button>
```

.ts

```
valider() {  
  alert("Voulez-vous vraiment valider?")  
}
```

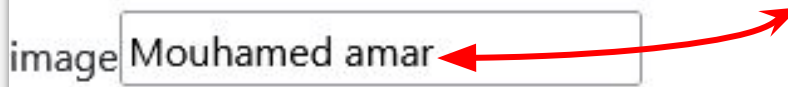
Liaison bidirectionnelle: **définition**

La liaison bidirectionnelle(**two way binding**) permet d'écouter les événements et mettre à jour les valeurs simultanément entre éléments HTML ou composants.

Liaison bidirectionnelle: **exemple**

Bonjour Mouhamed amar

image



.html

```
<h1>Bonjour {{prenom}}</h1>  
<input type="text" [(ngModel)]="prenom">
```



La propriété **ngModel** prends comme valeur la variable **prenom** et réagit au changement du champs de texte

Pipe: définition

Il permet de transformer des valeurs avant affichage dans votre modèle HTML. Une classe avec le décorateur **@Pipe** définit une fonction qui transforme les valeurs d'entrée en valeurs de sortie pour les afficher dans une vue.

Pipe: Examples

```
<h1>Je gagne {{salaire}} Fcfa par mois</h1>  
<h1>Je gagne {{salaire|number}} Fcfa par mois</h1>
```

Je gagne 1500000 Fcfa par mois
Je gagne 1,500,000 Fcfa par mois

Pipe: Exemples

Nom	Utilisation	Rôle
DatePipe	date	Formatage de date
UpperCasePipe	uppercase	Rendre en Majuscule
LoweCasePipe	lowercase	Rendre en Minuscule
CurrencyPipe	currency	Formatage en money
SlicePipe	slice	Filtrer
PercentPipe	percent	Afficher sous forme de %

Voir plus: <https://angular.io/api/common#pipes>

Directive: **NgIf**

Il inclut **conditionnellement** des éléments HTML en fonction de la valeur d'une **expression booléenne**.

- Si l'expression est évaluée à **true**, Angular **affiche** le modèle HTML
- Si l'expression est **false** ou **null**, le model HTML reste vide.

Directive: NgIf

`is_loading=true`

loading...

```
<h1 *ngIf="is_loading">loading...</h1>  
<h1 *ngIf="!is_loading">Chargement terminé</h1>
```

Directive: NgFor

Une directive structurelle qui affiche un ensemble d'éléments HTML pour chaque élément d'une **collection**. La directive est placée sur un élément, qui doit être répété.

Directive: NgFor

```
<ul>  
  <li *ngFor="let une_note of les_notes">  
    {{une_note}}  
  </li>  
</ul>
```

- 12
- 15
- 0.5
- 19
- 5.5

```
les_notes:number[]=[12,15,0.5,19,5.5]
```

Directive: NgClass

condition

succès

erreur

Une directive qui permet d'ajouter ou de retirer une ou plusieurs classes CSS à un élément HTML en fonction de conditions booléennes.

NgClass: example

.ts

```
status=1
```

.html

```
<div [ngClass]="{'success': status==1, 'error': status==-1}">
```

La variable status= {{status}}

```
</div>
```

.CSS

```
.success{  
  font-size: 5rem;  
  background-color: green;  
}
```

```
.error{  
  font-size: 5rem;  
  background-color: red;  
}
```

Partage de données: **définition**

Un modèle courant dans Angular est le partage de données entre un composant parent et un ou plusieurs composants enfants. Implémentez ce modèle avec les décorateurs **@Input()** et **@Output()**.

Partage de données: @Input()

app.component.html

```
<app-apropos [from_parent]="17">  
</app-apropos>
```

apropos.component.ts

```
@Input()  
from_parent:number=0
```

apropos.component.html

```
<h1>{{from_parent}}</h1>
```

Parent

Fils

Partage de données: @Output()

app.component.html

```
<app-ajout (to_parent)="reagir()">
</app-ajout>
```

ajout.component.ts

```
@Output()
to_parent=new EventEmitter<any>()
```

ajout.component.ts

```
this.to_parent.emit("data from child")
```

Parent

Fils



FIN

De chapitre

Pour plus de ressources, vous pouvez consulter:

<https://angular.io>

<https://blog.jant.tech>